

数据隔舱-底盘数据接口文档

接口格式说明

底盘数据接口分为状态类和控制类，二者均使用Redis Server实现数据的中转交互，其中状态类采用读取缓存的方式获取，控制类则通过发布订阅方式进行控制。数据内容使用protobuf直接序列化为字符串进行存取。

接口详细说明

云端控制指令

数据内容	Channel	值类型	备注
云端控制指令	CLOUD2VEH_REM OTECTL_1	CLOUD2VEH_REM OTECTL_1	

▼复制代码

```
message CLOUD2VEH_REMOTECTL_1 {  
  uint32 cloud_message_id = 1;  
  uint32 steering_angle = 2;           // 方向盘转角  
  uint32 steering_angular_velocity = 3; // 方向盘角速度  
  uint32 accel_pos = 4;                // 油门开度  
  uint32 brake_flag = 5;               // 制动踏板开关  
  uint32 brake_pos = 6;                // 制动踏板开度  
  uint32 tap_pos = 7;                  // 档位  
}
```

驱动状态报文

数据内容	Key	值类型	备注
驱动状态报文	DCU_Drive_St	DCU_Drive_St	

▼复制代码

```
message DCU_Drive_St {
```

```

enum DCU_AutoD_St {
    NOT_STARTED = 0;           // 未启动
    CONFIRM_STARTUP = 1;       // 确认启动
    ABNORMAL = 2;              // 异常
    INVALID = 3;               // 无效
}
DCU_AutoD_St dcu_autod_st = 1; // 智能驾驶确认信号
float dcu_accel_pos_value = 2; // 模拟加速踏板当前实际位置
float dcu_brkpel_pos_value = 3; // 模拟制动踏板当前实际位置
float dcu_real_speed = 4;      // 当前车速
uint32 dcu_life_signal = 5;    // 域控制器生命信号
}

```

域控制器基础信息 1

数据内容	Key	值类型	备注
域控制器基础信息 1	DCU_INFO_1	DCU_INFO_1	



复制代码

```

message DCU_INFO_1 {
    enum DCU_Current_Gear {
        GEAR_INVALID = 0;           // 无效
        R = 1;                     // 倒挡 R
        N = 2;                     // 空挡 N
        D = 3;                     // 前进挡 D
    }
    enum DCU_Veh_Drive_St {
        VEH_DRIVE_INVALID = 0;      // 无效
        POWER_ON_PROCESS = 1;       // 上电过程
        DRIVE_MODE = 2;             // 驱动模式
        LIMPING_MODE = 3;           // 跛行模式
        VEH_DRIVE_READY = 4;        // 准备就绪
        PREPARE_LAMENESS = 5;       // 跛行准备
        VEH_DRIVE_RESERVED_1 = 6;   // 预留
        VEH_DRIVE_RESERVED_2 = 7;   // 预留
    }
    enum DCU_Parking_St {
        RELEASE = 0;                // 释放
        PARKING = 1;                // 驻车
    }
    enum DCU_StrgPump_St {
        READY = 0;                 // 就绪
    }
}

```

```

    POWER_ON = 1;           // 上电
    FAULT = 2;              // 故障
    DIAGNOSIS = 3;         // 诊断
}
DCU_Current_Gear dcu_current_gear = 1;    // 当前档位
DCU_Veh_Drive_St dcu_veh_drive_st = 2;    // 车辆当前高压驱动状态
DCU_Parking_St dcu_parking_st = 3;        // 当前 P 档位状态
DCU_StrgPump_St dcu_strgpump_st = 4;      // 转向助力油泵状态
}

```

域控制器基础信息 2

数据内容	Key	值类型	备注
域控制器基础信息 2	DCU_INFO_2	DCU_INFO_2	



复制代码

```

message DCU_INFO_2 {
    enum AutoD_Limitin_Reason {
        NO_AUTOD_LIMITIN_REASON = 0; // 无
        EMERGENCY_STOP_BUTTON_TRIGGERED = 1; // 急停按钮触发
        REMOTE_PARKING_BUTTON_ACTIVATION = 2; // 远程停车按钮激活
        FRONT_COLLISION_TRIGGER = 3; // 前碰撞触发
        REAR_COLLISION_TRIGGER = 4; // 后碰撞触发
        SLOW_BRAKE_BUTTON_TRIGGERED = 5; // 缓刹按钮触发
        INTELLIGENT_DRIVING_BUTTON_NOT_SWITCHED = 6; // 智能驾驶按钮未切换
        INTELLIGENT_DRIVING_BUTTON_NOT_ALLOWED = 7; // 智能驾驶按钮未允许
        MECHANICAL_GEAR_NOT_SATISFIED = 8; // 机械档位不满足
        BRAKE_PEDAL_NOT_SATISFIED = 9; // 制动踏板不满足
        ACCELERATOR_PEDAL_NOT_SATISFIED = 10; // 油门踏板不满足
        STEERING_SYSTEM_MODE_NOT_SATISFIED = 11; // 转向系统模式不满足
        STEERING_WHEEL_ANGLE_NOT_SATISFIED = 12; // 方向盘实际角度不满足
        PARKING_STATUS_NOT_SATISFIED = 13; // 车辆驻车状态不满足
        PARKING_BUTTON_TRIGGERED = 14; // 驻车按钮触发
        SPEED_NOT_SATISFIED = 15; // 车速不满足
        HIGH_VOLTAGE_NOT_POWERED_ON = 16; // 高压未上电成功
        SELF_TEST_NOT_SATISFIED = 17; // 自检不满足
        LOW_VOLTAGE_SYSTEM_FAULTY = 18; // 低压系统故障
        BRAKE_HANDLE_NOT_SATISFIED = 19; // 缓刹手柄不满足
        REMOTE_CONTROL_HANDLE_TAKEOVER = 20; // 遥控手柄接管
        INTELLIGENT_SYSTEM_OFFLINE = 21; // 智能系统掉线
        GEAR_POSITION_REQUEST_NOT_MET = 22; // 档位请求不满足
        BRAKING_REQUEST_NOT_MET = 23; // 制动请求不满足
    }
}

```

```

    THROTTLE_REQUEST_NOT_MET = 24; // 油门请求不满足
    STEERING_WHEEL_ANGLE_REQUEST_NOT_MET = 25; // 方向盘角度请求不满足
    INTELLIGENT_DRIVING_COMMAND_NOT_SWITCHED = 26; // 智能驾驶指令未切换
    INTELLIGENT_SYSTEM_NOT_STARTED = 27; // 智能系统未启动
    INTELLIGENT_SYSTEM_STARTUP_INTERVAL_INSUFFICIENT = 28; // 智能系统启动时间间隔不满足
    SMART_PARKING_NOT_ALLOWED_TAKE_OVER = 29; // 智能停车不允许接管
    SEAT_BELT_NOT_FASTENED = 30; // 安全带未系
    DRIVER_LEAVES_SEAT = 31; // 司机离座
    DOOR_NOT_CLOSED = 32; // 门未关闭
}
enum Emergency_Stop_Reason {
    NO_EMERGENCY_STOP_REASON = 0; // 无
    EMERGENCY_STOP_SWITCH = 1; // 紧急停车开关
    REMOTE_EMERGENCY_STOP = 2; // 远程急停
    EMERGENCY_STOP_SWITCH_3 = 3; // 急停开关 3
    EMERGENCY_STOP_SWITCH_4 = 4; // 急停开关 4
    FRONT_COLLISION_TRIGGERED = 5; // 前碰撞触发
    AFTER_COLLISION_TRIGGER = 6; // 后碰撞触发
    EMERGENCY_STOP_REASON_RESERVED = 7; // 预留
}
enum DCU_Sys_LowFault_St {
    LOWFAULT_NO_FAULT = 0; // 无故障
    LOWFAULT_LEVEL_1_FAULT = 1; // 一级故障
    LOWFAULT_LEVEL_2_FAULT = 2; // 二级故障
    LOWFAULT_LEVEL_3_FAULT = 3; // 三级故障
    LOWFAULT_RESERVED_1 = 4; // 预留
    LOWFAULT_RESERVED_2 = 5; // 预留
    LOWFAULT_RESERVED_3 = 6; // 预留
    LOWFAULT_RESERVED_4 = 7; // 预留
}
enum DCU_Driver_TakeOver_Req {
    NOT_ACTIVATED = 0; // 未激活
    ACTIVATE_ALERT = 1; // 激活提醒
    TAKEOVER_RESERVED_1 = 2; // 预留
    TAKEOVER_RESERVED_2 = 3; // 预留
}
enum DCU_Vehicle_Drive_Mode_St {
    NO_DRIVE_MODE_ST = 0; // 无
    EMERGENCY_BRAKING_MODE = 1; // 紧急制动模式
    MANUAL_MODE = 2; // 手动模式
    DRIVE_MODE_ST_RESERVE = 3; // 预留
    AUTOMATIC_DRIVING_MODE = 4; // 自动驾驶模式
    SLOW_BRAKE_MODE = 5; // 缓刹模式
    SELF_CHECK_MODE = 6; // 自检模式
}

```

```

    REMOTE_DRIVING_MODE = 7; // 远程驾驶模式
}

enum AutoD_Out_Reason {
    NO_AUTOD_OUT_REASON = 0; // 无
    EMERGENCY_STOP_BUTTON_TRIGGERED_ = 1; // 急停按钮触发
    REMOTE_PARKING_BUTTON_ACTIVATION_ = 2; // 远程停车按钮激活
    FRONT_COLLISION_TRIGGER_ = 3; // 前碰撞触发
    REAR_COLLISION_TRIGGER_ = 4; // 后碰撞触发
    SLOW_BRAKE_BUTTON_TRIGGERED_ = 5; // 缓刹按钮触发
    INTELLIGENT_DRIVING_BUTTON_NOT_ALLOWED_ = 6; // 智能驾驶按钮未允许
    DRIVER_OPERATES_MECHANICAL_GEAR = 7; // 司机操作机械档位
    DRIVER_PRESSES_BRAKE_PEDAL = 8; // 司机踩制动踏板
    DRIVER_PRESSES_ACCELERATOR_PEDAL = 9; // 司机踩油门踏板
    DRIVER_TAKES_OVER_STEERING_WHEEL = 10; // 司机接管方向盘
    DRIVER_OPERATES_PARKING_BUTTON = 11; // 司机操作驻车按钮
    HIGH_VOLTAGE_POWER_OFF_STOP = 12; // 高压下电且停车
    LOW_PRESSURE_SERIOUS_FAILURE_STOP = 13; // 低压严重故障且停车
    DRIVER_OPERATES_BRAKE_HANDLE = 14; // 司机操作缓刹手柄
    HANDLE_HANDSHAKE_TAKEOVER = 15; // 手柄握手接管
    INTELLIGENT_SYSTEM_OFFLINE_ = 16; // 智能系统掉线
    INTELLIGENT_SYSTEM_EXITS = 17; // 智能系统退出
    STEERING_SYSTEM_MODE_NOT_SATISFIED_ = 18; // 转向系统模式不满足
}

enum MannulMode_Limitin_Reason {
    NO_MANNULMODE_LIMITIN_REASON = 0; // 无
    EMERGENCY_BRAKE_MODE_ACTIVATED = 1; // 紧急制动模式激活
    SLOW_BRAKE_MODE_ACTIVATED = 2; // 缓刹模式激活
    SELF_CHECK_NOT_COMPLETE = 3; // 自检未结束
    SLOW_BUTTON_ACTIVATION = 4; // 缓刹按钮激活
    INTELLIGENT_DRIVING_BUTTON_NOT_ALLOWED__ = 5; // 智能驾驶按钮未允许
    GEAR_REMOVAL = 6; // 档位移除
    SPEED_NOT_SATISFIED_ = 7; // 车速不满足
    HIGH_VOLTAGE_NOT_POWERED_ON_ = 8; // 高压未上电成功
    SMART_PARKING_NOT_ALLOWED_TAKE_OVER_ = 9; // 智能停车未允许接管
    OTHER_MANNULMODE_LIMITIN_REASON = 10; // 其他
}

enum ABS_Acive_St {
    ABS_NOT_ACTIVATED = 0; // 未激活
    ABS_ACTIVATE = 1; // 激活
    ABS_INVALID_1 = 2; // 无效
    ABS_INVALID_2 = 3; // 无效
}

enum Brak_Sys_Fault_St {
    BRAK_SYS_NO_FAULT = 0; // 无故障
    BRAK_SYS_LEVEL_1_FAULT = 1; // 一级故障
}

```

```

    BRAK_SYS_LEVEL_2_FAULT = 2;           // 二级故障
    BRAK_SYS_LEVEL_3_FAULT = 3;           // 三级故障
}
enum DCU_Brk_St {
    BRK_NOT_WORKING = 0;                  // 未工作
    BRK_AT_WORK = 1;                      // 工作中
    BRK_INVALID_1 = 2;                    // 无效
    BRK_INVALID_2 = 3;                    // 无效
}
enum DCU_Sys_HighFault_St {
    HIGHFAULT_NO_FAULT = 0;               // 无故障
    HIGHFAULT_LEVEL_1_FAULT = 1;          // 一级故障
    HIGHFAULT_LEVEL_2_FAULT = 2;          // 二级故障
    HIGHFAULT_LEVEL_3_FAULT = 3;          // 三级故障
    HIGHFAULT_LEVEL_4_FAULT = 4;          // 四级故障
    HIGHFAULT_INVALID_1 = 5;              // 无效
    HIGHFAULT_INVALID_2 = 6;              // 无效
    HIGHFAULT_INVALID_3 = 7;              // 无效
}
enum DCU_Start_Key_St {
    START_KEY_NOT_ACTIVATED = 0;          // 未激活
    START_KEY_ON = 1;                     // ON 档
    START_KEY_START = 2;                  // START 档
    START_KEY_ACC = 3;                    // ACC 档
}

AutoD_Limitin_Reason autod_limitin_reason = 1; // 限制进入自动驾驶原因
Emergency_Stop_Reason emergency_stop_reason = 2; // 紧急停车激活原因
DCU_Sys_LowFault_St dcu_sys_lowfault_st = 3; // 低压系统故障
DCU_Driver_TakeOver_Req dcu_driver_takeover_req = 4; // 驾驶员接管提醒
DCU_Vehicle_Drive_Mode_St dcu_vehicle_drive_mode_st = 5; // 车辆驾驶模式

AutoD_Out_Reason autod_out_reason = 6; // 退出自动驾驶原因
MannulMode_Limitin_Reason mannulmode_limitin_reason = 7; // 限制手动驾驶原因

ABS_Acive_St abs_acive_st = 8; // ABS 激活状态
Brak_Sys_Fault_St brak_sys_fault_st = 9; // 制动系统故障
DCU_Brk_St dcu_brk_st = 10; // 整车机械制动状态
DCU_Sys_HighFault_St dcu_sys_highfault_st = 11; // 高压系统故障
DCU_Start_Key_St dcu_start_key_st = 12; // 启动按键（点火开关）状态

```

高压电池状态报文

数据内容	Key	值类型	备注
------	-----	-----	----

高压电池状态报文	DCU_Battery_St	DCU_Battery_St	
----------	----------------	----------------	--

[复制代码](#)

```
message DCU_Battery_St {
    float Battery_Voltage = 1;           // 总电压
    float Battery_Current = 2;           // 总电流
    float Battery_SOC = 3;               // 高压电池电量
}
```

域控制器基础信息

数据内容	Key	值类型	备注
域控制器基础信息	DCU_INFO	DCU_INFO	

[复制代码](#)

```
message DCU_INFO {
    float driving_distance_left = 1;     // 剩余里程
}
```

转向系统状态反馈

数据内容	Key	值类型	备注
转向系统状态反馈	Strg_StrgSys_St	Strg_StrgSys_St	

[复制代码](#)

```
message Strg_StrgSys_St {
    enum Strg_WorkMode_St {
        STANDBY_MODE = 0;           // 待机模式
        AUTOMATIC_DRIVING_MODE = 1; // 自动驾驶模式
        WORKMODE_RESERVED = 2;      // 预留
        MOMENT_SUPERPOSITION_MODE = 3; // 力矩叠加模式
        MANUAL_MODE = 4;             // 手动模式
        MANUAL_INTERVENTION_MODE = 5; // 手动介入模式
        WARNING_MODE = 6;            // 警告模式
        ERROR_MODE = 7;              // 错误模式（故障模式）
        WORKMODE_RESERVED_1 = 8;     // 预留
    }
}
```

```

    WORKMODE_RESERVED_2 = 9;           // 预留
    WORKMODE_RESERVED_3 = 10;          // 预留
    WORKMODE_RESERVED_4 = 11;          // 预留
    WORKMODE_RESERVED_5 = 12;          // 预留
    WORKMODE_RESERVED_6 = 13;          // 预留
    WORKMODE_RESERVED_7 = 14;          // 预留
    WORKMODE_RESERVED_8 = 15;          // 预留
}
float strg_torq_value = 1;             // 方向盘当前实际扭矩反馈
float strg_motor_torq_value = 2;       // 电机助力扭矩
float strg_angle_real_value = 3;       // 目标方向盘转向实际角度反馈
uint32 strg_angle_spd_value = 4;       // 方向盘当前实际速度反馈
Strg_WorkMode_St strg_workmode_st = 5; // 当前系统实际工作模式
uint32 strg_life_signal = 6;           // life 生命脉动
}

```

L2 功能状态反馈

数据内容	Key	值类型	备注
L2 功能状态反馈	DCU_L2_St	DCU_L2_St	



复制代码

```

message DCU_L2_St {
    enum ACC_Work_Status {
        ACC_INITIALIZATION = 0; // 初始化
        ACC_CLOSE = 1;          // 关闭
        ACC_ENABLE = 2;         // 开启
        ACC_STANDBY = 3;        // 待机
        ACC_ACTIVATED = 4;      // 激活
        ACC_BEYOND_CONTROL = 5; // 超越控制
        ACC_SUPPRESS = 6;       // 抑制
        ACC_FAULT = 7;          // 故障
    }
    enum ACC_Work_Mode {
        ACC_WORK_MODE_NONE = 0; // 无
        CONSTANT_SPEED_CONTROL = 1; // 定速控制
        FOLLOW_CONTROL = 2; // 跟随控制
        PARKING_CONTROL = 3; // 停车控制
        START_CONTROL = 4; // 启动控制
        ACC_WORK_MODE_INVALID_1 = 5; // 无效
        ACC_WORK_MODE_INVALID_2 = 6; // 无效
        ACC_WORK_MODE_INVALID_3 = 7; // 无效
    }
}

```



```

}
enum ACC_Fail_Reason {
    ACC_FAIL_REASON_NONE = 0;           // 无不满足条件
    GEAR_POSITION_NOT_SATISFIED = 1;     // 档位不满足
    HAND_BRAKE_NOT_RELEASED = 2;        // 手刹未松开
    BRAKE_NOT_RELEASED = 3;             // 刹车未松开
    THROTTLE_OPENING_GREATER_60_PERCENT = 4; // 油门开度大于 60%
    VEHICLE_MODE_NOT_SATISFIED = 5;     // 车辆模式不满足
    ACC_SYSTEM_SELF_CHECK_ERROR = 6;    // ACC 系统自检错误
    KEY_PRIORITY_OPERATION_OFF = 7;     // 按键（优先） 操作关
闭
    WIPER_OPENS_MEDIUM_HIGH_SPEED_GEAR = 8; // 雨刮开启中高速档位
    LANE_WIDTH_NOT_SATISFIED = 9;       // 车道宽度不满足
    LANE_CONFIDENCE_LEVEL_NOT_SATISFIED = 10; // 车道线置信度不满足
    ROAD_CURVATURE_RADIUS_NOT_SATISFIED = 11; // 道路曲率半径不满足
}
enum ACC_Active2Quit_Reason {
    ACC_ACTIVE2QUIT_REASON_NONE = 0;    // 无不满足条件
    ACC_SELF_TEST_FAILED = 1;           // ACC 自检未通过
    HANDBRAKE_ABNORMALLY_PULLED_UP = 2; // 车辆手刹异常拉起
    MODE_NOT_SATISFIED_ = 3;            // 车辆模式不满足
    GEAR_NOT_IN_FORWARD_GEAR = 4;       // 车辆档位不处于前进
挡
    ACC_CANCEL = 5;                     // ACC_CANCEL
    WIPER_OPEN_IN_THE_MIDDLE_AND_HIGH_LEVEL = 6; // 雨刮开启中高档位
    LANE_WIDTH_NOT_SATISFIED_ = 7;      // 车道宽不满足
    LANE_CONFIDENCE_LEVEL_NOT_SATISFIED_ = 8; // 车道线置信度不满足
    ROAD_CURVATURE_RADIUS_NOT_SATISFIED_ = 9; // 道路曲率半径不满足
    THROTTLE_OPENING_GREATER_THRESHOLD = 10; // 油门开度大于阈值且
持续一定时间
    DRIVER_BRAKES = 11;                 // 司机踩制动
    SPEED_LESS_ACC_MINIMUM_SPEED = 12;  // 实际车速小于 ACC 允
许最低车速
    COOPERATIVE_CONTROLLER_FAILURE = 13; // 协同控制器失效
}
enum LKA_Work_Status {
    LKA_INVALID = 0; // 无效
    LKA_CLOSE = 1;   // 关闭
    LKA_STANDBY = 2;  // 待机
    LKA_ACTIVATED = 3; // 激活
    LKA_EXIT = 4;     // 退出
    LKA_FAULT = 5;    // 故障
    LKA_INVALID_1 = 6; // 无效
    LKA_INVALID_2 = 7; // 无效
}

```

```

enum LKA_Active2Quit_Reason {
    LKA_ACTIVE2QUIT_REASON_NONE = 0;           // 无不满足条件
    VEHICLE_MODE_NOT_SATISFIED__ = 1;          // 车辆模式不满足
    STEERING_WHEEL_TORQUE_GREATER_THRESHOLD = 2; // 方向盘当前实际扭矩
    CMS_FUNCTION_ACTIVATED = 3;                 // 前向防撞 CMS 功能激活
    ACC_FUNCTION_EXITS = 4;                    // ACC 功能退出
    TURN_SIGNAL_ON = 5;                        // 转向灯开启
    DRIVER_FULL_THROTTLE_OUTPUT = 6;           // 驾驶员油门输出
    DEEP_BRAKING = 7;                          // 深踩制动
    DRIVER_AWAY_SEAT_EXCEEDS_THRESHOLD = 8;    // 驾驶员离座持续时间超过一定阈值
    WIPER_OPEN_MIDDLE_AND_HIGH_LEVEL = 9;      // 雨刮开启中高档位
    LANE_WIDTH_INVALID = 10;                   // 车道宽无效
    LANE_CONFIDENCE_LEVEL_NOT_SATISFIED__ = 11; // 车道线置信度不满足
    ROAD_CURVATURE_RADIUS_LESS_THRESHOLD = 12; // 道路曲率半径小于阈值
    SPEED_EXCEEDS_MAXIMUM_SPEED_LIMIT = 13;    // 车速超过最高限速
    ACCELERATION_EXCEEDS_SAFETY_THRESHOLD = 14; // 横纵向加速度超过安全阈值
    KEY_OPERATION_OFF = 15;                    // 按键（优先）操作关闭
    COOPERATIVE_CONTROLLER_FAILURE_ = 16;       // 协同控制器失效
}

enum LKA_Fail_Reason {
    LKA_FAIL_REASON_NONE = 0;                 // 无不满足条件
    VEHICLE_GEAR_NOT_SATISFIED = 1;            // 车辆档位不满足
    STEERING_WHEEL_TORQUE_NOT_SATISFIED = 2;   // 方向盘当前实际扭矩不满足
    DOOR_NOT_CLOSED = 3;                      // 车门未关
    ACTUAL_STEERING_ANGLE_NOT_SATISFIED = 4;    // 方向盘转向实际角度不满足
    ACC_NOT_ACTIVATED = 5;                    // ACC 未激活
    TURN_SIGNAL_ON_ = 6;                      // 转向灯开启
    ACCELERATOR_PEDAL_NOT_SATISFIED = 7;       // 油门踏板不满足
    BRAKE_PEDAL_NOT_SATISFIED = 8;             // 制动踏板不满足
    DRIVER_LEAVES_SEAT = 9;                   // 司机离座
    WIPER_OPENS_MEDIUM_HIGH_SPEED_GEAR_ = 10;  // 雨刮开启中高速档位
    LANE_WIDTH_NOT_SATISFIED__ = 11;           // 车道宽度不满足
    LANE_LINE_CONFIDENCE_NOT_SATISFIED = 12;   // 车道线置信度不满足
}

```

```

ROAD_CURVATURE_RADIUS_NOT_SATISFIED__ = 13;           // 道路曲率半径不满
是
SPEED_NOT_SATISFIED = 14;                             // 车速不满足
ACCELERATION_NOT_SATISFIED = 15;                       // 横/纵向加速度不满
是
HANDBRAKE_NOT_RELEASED = 16;                           // 手刹未释放
VEHICLE_MODE_NOT_SATISFIED__ = 17;                     // 车辆模式不满足
}
enum ADAS_LDW_Status {
    ADAS_INVALID = 0;           // 无效
    ADAS_CLOSE = 1;            // 关闭
    ADAS_STANDBY = 2;          // 待机
    ADAS_ACTIVATED = 3;        // 激活
    ADAS_FAULT = 4;            // 故障
    ADAS_INVALID_1 = 5;        // 无效
    ADAS_INVALID_2 = 6;        // 无效
    ADAS_INVALID_3 = 7;        // 无效
}
enum L2_Function_Active_Model {
    L2_NOT_ACTIVATED = 0;           // 未激活
    LKA_FUNCTION_ACTIVATED = 1;     // LKA 横向控制功能激活
    ACC_FUNCTION_ACTIVATED = 2;     // ACC 纵向控制功能激活
    L2_FUNCTION_ACTIVATED = 3;      // L2 功能激活
    CMS_AEB_FUNCTION_ACTIVATED = 4; // CMS&AEB 功能激活
    ADAS_RESERVED_1 = 5;           // 预留
    ADAS_RESERVED_2 = 6;           // 预留
    ADAS_RESERVED_3 = 7;           // 预留
}
enum CMS_Status {
    CMS_INITIALIZATION = 0;        // 初始化
    CMS_CLOSE = 1;                 // 关闭
    CMS_ENABLE = 2;                // 开启
    CMS_ACTIVATED = 3;             // 激活
    CMS_FAULT = 4;                 // 故障
    CMS_RESERVED_1 = 5;            // 预留
    CMS_RESERVED_2 = 6;            // 预留
    CMS_RESERVED_3 = 7;            // 预留
}
enum CMS_Warning_Level {
    CMS_WARNING_LEVEL_NOT_ACTIVATED = 0; // 未生效
    DISTANCE_WARNING = 1; // 车距预警（车辆一级碰撞预警）
    COLLISION_WARNING = 2; // 碰撞预警（车辆二级碰撞预警）
    VEHICLE_CMS_EMERGENCY_BRAKE_ACTIVATED = 3; // 车辆 CMS&紧急制动激活
    PEDESTRIAN_COLLISION_WARNING_LEVEL_1 = 4; // 行人一级碰撞预警
    PEDESTRIAN_COLLISION_WARNING_LEVEL_2 = 5; // 行人二级碰撞预警
}

```

```

PEDESTRIAN_CMS_EMERGENCY_BRAKE_ACTIVATED = 6; // 行人 CMS&紧急制动激活
CMS_WARNING_LEVEL_INVALID = 7; // 无效
}

ACC_Work_Status acc_work_status = 1; // ACC 工作状态
ACC_Work_Mode acc_work_mode = 2; // ACC 工作模式
ACC_Fail_Reason acc_fail_reason = 3; // ACC 开启且未达到功能启动条件原因
ACC_Active2Quit_Reason acc_active2quit_reason = 4; // ACC 功能激活退出原因
LKA_Work_Status lka_work_status = 5; // LKA 工作状态
LKA_Active2Quit_Reason lka_active2quit_reason= 6; // LKA 功能激活退出原因
LKA_Fail_Reason lka_fail_reason = 7; // LKA 开启且未达到功能启动条件原因
ADAS_LDW_Status adas_ldw_status = 8; // LDW 工作状态
L2_Function_Active_Model l2_function_active_model = 9; // L2 功能激活模式

```

智能控制状态反馈

数据内容	Key	值类型	备注
智能控制状态反馈	BC_Intelligent_Ctrl_St	BC_Intelligent_Ctrl_St	



复制代码

```

message BC_Intelligent_Ctrl_St {
    enum BC_Emergence_Key_St {
        BC_EMERGENCE_KEY_NOT_ACTIVATED = 0; // 未激活
        BC_EMERGENCE_KEY_ACTIVATE = 1; // 激活
        BC_EMERGENCE_KEY_RESERVED_1 = 2; // 预留
        BC_EMERGENCE_KEY_RESERVED_2 = 3; // 预留
    }
    enum BC_AutoD_Out_Key_St {
        BC_AUTOD_OUT_KEY_NOT_ACTIVATED = 0; // 未激活
        BC_AUTOD_OUT_KEY_ACTIVATE = 1; // 激活
        BC_AUTOD_OUT_KEY_RESERVED_1 = 2; // 预留
        BC_AUTOD_OUT_KEY_RESERVED_2 = 3; // 预留
    }
    enum BC_Start_Key_St {
        BC_START_KEY_NOT_ACTIVATED = 0; // 未激活
        BC_START_KEY_ON = 1; // ON 档
        BC_START_KEY_START = 2; // START 档
        BC_START_KEY_ACC = 3; // ACC 档
    }
}

```

```

}
enum BC_SafetyBelt_Alarm_St {
    BC_SAFETYBELT_ALARM_NOT_ACTIVATED = 0;    // 未激活
    BC_SAFETYBELT_ALARM_ACTIVATE = 1;         // 激活
    BC_SAFETYBELT_ALARM_RESERVED_1 = 2;       // 预留
    BC_SAFETYBELT_ALARM_RESERVED_2 = 3;       // 预留
}
enum BC_DriverLeft_Alarm_St {
    BC_DRIVERLEFT_ALARM_NOT_ACTIVATED = 0;    // 未激活
    BC_DRIVERLEFT_ALARM_ACTIVATE = 1;         // 激活
    BC_DRIVERLEFT_ALARM_RESERVED_1 = 2;       // 预留
    BC_DRIVERLEFT_ALARM_RESERVED_2 = 3;       // 预留
}
enum BC_In_FDoor_Key_St {
    BC_IN_FDOOR_KEY_NOT_ACTIVATED = 0;        // 未激活
    BC_IN_FDOOR_KEY_ACTIVATE_DOOR_OPEN = 1;   // 激活开门
    BC_IN_FDOOR_KEY_ACTIVATE_DOOR_CLOSE = 2;  // 预留关门
    BC_IN_FDOOR_KEY_RESERVED_1 = 3;           // 预留
}
enum BC_Out_FDoor_Key_St {
    BC_OUT_FDOOR_KEY_NOT_ACTIVATED = 0;        // 未激活
    BC_OUT_FDOOR_KEY_ACTIVATE = 1;             // 激活
    BC_OUT_FDOOR_KEY_RESERVED_1 = 2;           // 预留
    BC_OUT_FDOOR_KEY_RESERVED_2 = 3;           // 预留
}
enum BC_Horn_Key_St {
    BC_HORN_KEY_NOT_ACTIVATED = 0;             // 未激活
    BC_HORN_KEY_ACTIVATE = 1;                  // 激活
    BC_HORN_KEY_NULL = 2;                      // NULL
    BC_HORN_KEY_RESERVED_1 = 3;                // 预留
}
enum BC_Defrosting_St {
    BC_DEFROSTING_NOT_OUTPUT = 0;              // 未输出
    BC_DEFROSTING_LOW_OUTPUT = 1;              // 低档输出
    BC_DEFROSTING_HIGH_OUTPUT = 2;             // 高档输出
    BC_DEFROSTING_INVALID = 3;                 // 无效
}
enum BC_Hazard_Warning_Light_St {
    BC_HAZARD_WARNING_LIGHT_NOT_ACTIVATED = 0; // 未激活
    BC_HAZARD_WARNING_LIGHT_ACTIVATE = 1;      // 激活
    BC_HAZARD_WARNING_LIGHT_RESERVED_1 = 2;    // 预留
    BC_HAZARD_WARNING_LIGHT_RESERVED_2 = 3;    // 预留
}
enum BC_In_MDoor_Key_St {
    BC_IN_MDOOR_KEY_NOT_ACTIVATED = 0;         // 未激活

```

```

BC_IN_MDOOR_KEY_ACTIVATE_DOOR_OPEN = 1;    // 激活开门
BC_IN_MDOOR_KEY_ACTIVATE_DOOR_CLOSE = 2;    // 预留关门
BC_IN_MDOOR_KEY_RESERVED_1 = 3;            // 预留
}
enum BC_Out_MDoor_Key_St {
    BC_OUT_MDOOR_KEY_NOT_ACTIVATED = 0;      // 未激活
    BC_OUT_MDOOR_KEY_ACTIVATE = 1;          // 激活
    BC_OUT_MDOOR_KEY_RESERVED_1 = 2;        // 预留
    BC_OUT_MDOOR_KEY_RESERVED_2 = 3;        // 预留
}
enum BC_In_RDoor_Key_St {
    BC_IN_RDOOR_KEY_NOT_ACTIVATED = 0;      // 未激活
    BC_IN_RDOOR_KEY_ACTIVATE_DOOR_OPEN = 1;  // 激活开门
    BC_IN_RDOOR_KEY_ACTIVATE_DOOR_CLOSE = 2; // 预留关门
    BC_IN_RDOOR_KEY_RESERVED_1 = 3;          // 预留
}
enum BC_Out_RDoor_Key_St {
    BC_OUT_RDOOR_KEY_NOT_ACTIVATED = 0;      // 未激活
    BC_OUT_RDOOR_KEY_ACTIVATE = 1;          // 激活
    BC_OUT_RDOOR_KEY_RESERVED_1 = 2;        // 预留
    BC_OUT_RDOOR_KEY_RESERVED_2 = 3;        // 预留
}

BC_Emergence_Key_St bc_emergence_key_st = 1; // 紧急危险按键状态
BC_AutoD_Out_Key_St bc_autod_out_key_st = 2; // 屏蔽智能驾驶按键状态
BC_Start_Key_St bc_start_key_st = 3; // 启动按键（点火开关）状态
BC_SafetyBelt_Alarm_St bc_safetybelt_alarm_st = 4; // 司机安全带未系报警
BC_DriverLeft_Alarm_St bc_driverleft_alarm_st = 5; // 司机离座报警
BC_In_FDoor_Key_St bc_in_fdoor_key_st = 6; // 车内前门按键状态
BC_Out_FDoor_Key_St bc_out_fdoor_key_st = 7; // 车外前门按键状态
BC_Horn_Key_St bc_horn_key_st = 8; // 喇叭按键状态
BC_Defrosting_St bc_defrosting_st = 9; // 除霜状态
BC_Hazard_Warning_Light_St bc_hazard_warning_light_st = 10; // 危险警告
信号按键状态
BC_In_MDoor_Key_St bc_in_mdoor_key_st = 11; // 车内中门按键状态
BC_Out_MDoor_Key_St bc_out_mdoor_key_st = 12; // 车外中门控制按键状态
BC_In_RDoor_Key_St bc_in_rdoor_key_st = 13; // 车内后门按键状态
BC_Out_RDoor_Key_St bc_out_rdoor_key_st = 14; // 车外后门控制按键状态
}

```

车辆总里程报文

数据内容	Key	值类型	备注
------	-----	-----	----

车辆总里程报文	BC_Veh_Mileage	BC_Veh_Mileage	
---------	----------------	----------------	--

[复制代码](#)

```
message BC_Veh_Mileage {
    float bc_mileage_short = 1;    // 短里程
    float bc_mileage_long = 2;    // 长里程
}
```

智能车辆状态反馈

数据内容	Key	值类型	备注
智能车辆状态反馈	BC_AutoD_Veh_St	BC_AutoD_Veh_St	

[复制代码](#)

```
message BC_AutoD_Veh_St {
    enum BC_Veh_DataPackage_NO {
        MEANINGLESS = 0;    // 未激活
        PACKET_NUMBER_1 = 1;    // 第 1 号数据包
        PACKET_NUMBER_2 = 2;    // 第 2 号数据包
        PACKET_NUMBER_3 = 3;    // 第 3 号数据包
    }
    enum BC_ReversingLight_St {
        REVERSINGLIGHT_CLOSE = 0;    // 关闭
        REVERSINGLIGHT_OPEN = 1;    // 开启
        REVERSINGLIGHT_RESERVED_1 = 2;    // 预留
        REVERSINGLIGHT_RESERVED_2 = 3;    // 预留
    }
    enum BC_Front_Door_St {
        FRONT_DOOR_INVALID = 0;    // 无效
        FRONT_DOOR_OPEN = 1;    // 门已经开
        FRONT_DOOR_CLOSED = 2;    // 门已经关
        FRONT_DOOR_OPENING = 3;    // 门正在开的过程
        FRONT_DOOR_CLOSING = 4;    // 门正在关的过程
        FRONT_DOOR_RESERVED_1 = 5;    // 预留
        FRONT_DOOR_RESERVED_2 = 6;    // 预留
        FRONT_DOOR_RESERVED_3 = 7;    // 预留
    }
    enum BC_Mid_Door_St {
        MID_DOOR_INVALID = 0;    // 无效
        MID_DOOR_OPEN = 1;    // 门已经开
    }
}
```

```

MID_DOOR_CLOSED = 2;           // 门已经关
MID_DOOR_OPENING = 3;          // 门正在开的过程
MID_DOOR_CLOSING = 4;          // 门正在关的过程
MID_DOOR_RESERVED_1 = 5;       // 预留
MID_DOOR_RESERVED_2 = 6;       // 预留
MID_DOOR_RESERVED_3 = 7;       // 预留
}

enum BC_BrkLight_St {
    BRKLIGHT_CLOSE = 0;         // 关闭
    BRKLIGHT_OPEN = 1;          // 开启
    BRKLIGHT_RESERVED_1 = 2;     // 预留
    BRKLIGHT_RESERVED_2 = 3;     // 预留
}

enum BC_DayLight_2_St {
    DAYLIGHT_2_CLOSE = 0;       // 关闭
    DAYLIGHT_2_OPEN = 1;        // 开启
    DAYLIGHT_2_RESERVED_1 = 2;   // 预留
    DAYLIGHT_2_RESERVED_2 = 3;   // 预留
}

enum BC_BrkOilLevel_Alarm {
    BRKOILLEVEL_NO_ALARM = 0;    // 未报警
    BRKOILLEVEL_ALARM = 1;       // 报警
    BRKOILLEVEL_RESERVED_1 = 2;   // 预留
    BRKOILLEVEL_RESERVED_2 = 3;   // 预留
}

enum BC_Horn_St {
    HORN_CLOSE = 0;             // 关闭
    HORN_OPEN = 1;              // 开启
    HORN_RESERVED_1 = 2;        // 预留
    HORN_RESERVED_2 = 3;        // 预留
}

enum BC_DayLight_1_St {
    DAYLIGHT_1_CLOSE = 0;       // 关闭
    DAYLIGHT_1_OPEN = 1;        // 开启
    DAYLIGHT_1_RESERVED_1 = 2;   // 预留
    DAYLIGHT_1_RESERVED_2 = 3;   // 预留
}

enum BC_Front_Foglamp_St {
    FRONT_FOGLAMP_CLOSE = 0;    // 关闭
    FRONT_FOGLAMP_OPEN = 1;     // 开启
    FRONT_FOGLAMP_RESERVED_1 = 2; // 预留
    FRONT_FOGLAMP_RESERVED_2 = 3; // 预留
}

enum BC_Rear_Foglamp_St {
    REAR_FOGLAMP_CLOSE = 0;     // 关闭

```



```

    REAR_FOGLAMP_OPEN = 1;           // 开启
    REAR_FOGLAMP_RESERVED_1 = 2;     // 预留
    REAR_FOGLAMP_RESERVED_2 = 3;     // 预留
}
enum BC_Bean_Light_St {
    BEAN_LIGHT_CLOSE = 0;           // 关闭
    BEAN_LIGHT_OPEN = 1;            // 开启
    BEAN_LIGHT_RESERVED_1 = 2;      // 预留
    BEAN_LIGHT_RESERVED_2 = 3;      // 预留
}
enum BC_LowBeam_St {
    LOWBEAM_CLOSE = 0;             // 关闭
    LOWBEAM_OPEN = 1;              // 开启
    LOWBEAM_RESERVED_1 = 2;        // 预留
    LOWBEAM_RESERVED_2 = 3;        // 预留
}
enum BC_TurnRightLight_St {
    TURNRIGHTLIGHT_CLOSE = 0;      // 关闭
    TURNRIGHTLIGHT_OPEN = 1;       // 开启
    TURNRIGHTLIGHT_RESERVED_1 = 2;  // 预留
    TURNRIGHTLIGHT_RESERVED_2 = 3;  // 预留
}
enum BC_TurnLeftLight_St {
    TURNLEFTLIGHT_CLOSE = 0;       // 关闭
    TURNLEFTLIGHT_OPEN = 1;        // 开启
    TURNLEFTLIGHT_RESERVED_1 = 2;   // 预留
    TURNLEFTLIGHT_RESERVED_2 = 3;   // 预留
}
enum BC_DriverLight_St {
    DRIVERLIGHT_CLOSE = 0;         // 关闭
    DRIVERLIGHT_OPEN = 1;          // 开启
    DRIVERLIGHT_RESERVED_1 = 2;     // 预留
    DRIVERLIGHT_RESERVED_2 = 3;     // 预留
}
enum BC_Minilight_St {
    MINILIGHT_CLOSE = 0;          // 关闭
    MINILIGHT_OPEN = 1;           // 开启
    MINILIGHT_RESERVED_1 = 2;      // 预留
    MINILIGHT_RESERVED_2 = 3;      // 预留
}
enum BC_BackDoor_Key_St {
    BACKDOOR_CLOSE = 0;           // 关闭
    BACKDOOR_OPEN = 1;            // 开启
    BACKDOOR_RESERVED_1 = 2;       // 预留
    BACKDOOR_RESERVED_2 = 3;       // 预留
}

```

```

}
enum BC_RoofLight_St {
    ROOFLIGHT_CLOSE = 0;           // 关闭
    ROOFLIGHT_OPEN = 1;            // 开启
    ROOFLIGHT_RESERVED_1 = 2;      // 预留
    ROOFLIGHT_RESERVED_2 = 3;      // 预留
}
enum BC_CoolWater_Level_Alarm_St {
    COOLWATER_LEVEL_NO_ALARM = 0;  // 未报警
    COOLWATER_LEVEL_ALARM = 1;     // 报警
    COOLWATER_LEVEL_RESERVED_1 = 2; // 预留
    COOLWATER_LEVEL_RESERVED_2 = 3; // 预留
}
enum BC_Emergency_Enable_Alarm {
    EMERGENCY_ENABLE_NO_ALARM = 0; // 未报警
    EMERGENCY_ENABLE_ALARM = 1;     // 报警
    EMERGENCY_ENABLE_RESERVED_1 = 2; // 预留
    EMERGENCY_ENABLE_RESERVED_2 = 3; // 预留
}
enum BC_BrkPad_Worn_St {
    BRKPAD_WORN_NO_ALARM = 0;       // 未报警
    BRKPAD_WORN_ALARM = 1;          // 报警
    BRKPAD_WORN_RESERVED_1 = 2;     // 预留
    BRKPAD_WORN_RESERVED_2 = 3;     // 预留
}
enum BC_Wiper_St {
    WIPER_OFF = 0;                  // 未报警
    WIPER_INTERMITTENT = 1;         // 报警
    WIPER_LOW_SPEED = 2;            // 预留
    WIPER_HIGH_SPEED = 3;           // 预留
}
enum BC_EmergDoorSw_Alarm {
    EMERGDOORSW_NO_ALARM = 0;       // 未报警
    EMERGDOORSW_ALARM = 1;          // 报警
    EMERGDOORSW_RESERVED_1 = 2;     // 预留
    EMERGDOORSW_RESERVED_2 = 3;     // 预留
}
BC_Veh_DataPackage_NO bc_veh_datapackage_no = 1; // 标识符（当前数据包号）
BC_ReversingLight_St bc_reversinglight_st = 2; // 倒车灯状态
BC_Front_Door_St bc_front_door_st = 3; // 前门状态
BC_Mid_Door_St bc_mid_door_st = 4; // 中门状态
BC_BrkLight_St bc_brklight_st = 5; // 制动灯状态
BC_DayLight_2_St bc_daylight_2_st = 6; // 箱灯 2 或日光灯 2 状态
BC_BrkOilLevel_Alarm bc_brkoillevel_alarm = 7; // 制动液液位低报警

```

```

BC_Horn_St bc_horn_st = 8; // 喇叭状态
BC_DayLight_1_St bc_daylight_1_st = 9; // 箱灯 1 或日光灯 1 状态
BC_Front_Foglamp_St bc_front_foglamp_st = 10; // 前雾灯状态
BC_Rear_Foglamp_St bc_rear_foglamp_st = 11; // 后雾灯状态
BC_Bean_Light_St bc_bean_light_st = 12; // 远光灯状态
BC_LowBeam_St bc_lowbeam_st = 13; // 近光灯状态
BC_TurnRightLight_St bc_turnrightlight_st = 14; // 右转向灯状态
BC_TurnLeftLight_St bc_turnleftlight_st = 15; // 左转向灯状态
BC_DriverLight_St bc_driverlight_st= 16; // 司机灯状态
BC_MiniLight_St bc_minilight_st = 17; // 小灯状态（包括示廓灯
BC_BackDoor_Key_St bc_backdoor_key_st = 18; // 后舱门状态信号
BC_RoofLight_St bc_rooflight_st = 19; // 踏步灯或门顶灯状态
BC_CoolWater_Level_Alarm_St bc_coolwater_level_alarm_st = 20; // 冷却水位过低报警-水冷系统
BC_Emergency_Enable_Alarm bc_emergency_enable_alarm = 21; // 应急阀打开报警
BC_BrkPad_Worn_St bc_brkpad_worn_st = 22; // 刹车蹄片磨损状态
BC_Wiper_St bc_wiper_st = 23; // 雨刮状态
BC_FrontDoorKey_Alarm bc_frontdoorkey_alarm = 24; // 前车门应急阀开关报警

```

车辆车速报文

数据内容	Key	值类型	备注
车辆车速报文	BC_Veh_Spd	BC_Veh_Spd	



复制代码

```

message BC_Veh_Spd {
    float bc_veh_spd_value = 1; // 车速
}

```

轮速报文

数据内容	Key	值类型	备注
驱动状态报文	Rel_SpeedSteer	Rel_SpeedSteer	



复制代码

```

message Rel_SpeedSteer {
    float steeraxlespeed = 1; // 前轴速度
    float rel_speedsteeraxlelef = 2; // 前左轮相对速度
}

```

```

float rel_speedsteeraxleright = 3;    // 前右轮相对速度
float rel_speedrearaxleleft = 4;      // 第二轴左轮相对速度
float rel_speedrearaxleright = 5;     // 第二轴右轮相对速度
float rel_speedthirdaxleleft = 6;     // 第三轴左轮相对速度
float rel_speedthirdaxleright = 7;    // 第三轴右轮相对速度
}

```

胎压监控系统状态

数据内容	Key	值类型	备注
胎压监控系统状态	TPMS_INFO	TPMS_INFO	



复制代码

```

message TPMS_INFO {
    enum Tire_Location {
        AXIS_1_FETUS_1 = 0;    // 轴 1 胎 1
        AXIS_1_FETUS_2 = 1;    // 轴 1 胎 2
        AXIS_2_FETUS_1 = 16;   // 轴 2 胎 1
        AXIS_2_FETUS_2 = 17;   // 轴 2 胎 2
        AXIS_2_FETUS_3 = 18;   // 轴 2 胎 3
        AXIS_2_FETUS_4 = 19;   // 轴 2 胎 4
        AXIS_3_FETUS_1 = 32;   // 轴 3 胎 1
        AXIS_3_FETUS_2 = 33;   // 轴 3 胎 1
        SPARE_TIRE_1 = 96;     // 备胎 1
        SPARE_TIRE_2 = 97;     // 备胎 2
    }
    enum Tire_HighTemp_Alarm {
        HIGHTEMP_NORMAL = 0;   // 正常
        HIGHTEMP_ALARM = 1;    // 高温报警
        HIGHTEMP_RESERVED_1 = 2; // 保留
        HIGHTEMP_RESERVED_2 = 3; // 保留
    }
    enum Tire_Leak_Alarm {
        LEAK_NORMAL = 0;       // 正常
        LEAK_ALARM = 1;        // 轮胎漏气
        LEAK_RESERVED_1 = 2;    // 保留
        LEAK_RESERVED_2 = 3;    // 保留
    }
    enum Tire_Lost_Alarm {
        LOST_NORMAL = 0;       // 正常
        SENSOR_FAULT_ALARM = 1; // 传感器故障报警
        LOST_RESERVED_1 = 2;    // 保留
    }
}

```

```

    LOST_RESERVED_2 = 3;           // 保留
}
enum Tire_Press_Valve_Alarm {
    EXCEEDS_LIMIT_PRESSURE = 0;    // 超出极限压力
    OVERVOLTAGE = 1;               // 关闭过压
    NO_PRESSURE_ALARM = 2;         // 没有压力报警
    PRESSURE_TOO_LOW = 3;          // 压力过低
    BELOW_LOW_PRESSURE_LIMIT = 4;   // 低于低压极限
    PRESS_VALVE_INVALID = 5;        // 无效值
    INDICATOR_ERROR = 6;           // 指示器错误
    PRESS_VALVE_RESERVED = 7;      // 保留
}

Tire_Location tire_location = 1;   // 轮胎定位
uint32 Tire_Pressure = 2;          // 轮胎压力
float Tire_Temp = 3;               // 轮胎温度
Tire_HighTemp_Alarm tire_hightemp_alarm = 4; // 轮胎高温报警
Tire_Leak_Alarm tire_leak_alarm = 5; // 轮胎漏气报警
Tire_Lost_Alarm tire_lost_alarm = 6; // 轮胎信号丢失报警
Tire_Press_Valve_Alarm tire_press_valve_alarm = 7; // 轮胎压力阈值报警
}

```

智能车辆 VIN 码

数据内容	Key	值类型	备注
智能车辆 VIN 码	VIN_INFO	VIN_INFO	



复制代码

```

message VIN_INFO {
    uint32 vin_datapackage_no = 1; // 标识符 n (当前数据包号)
    uint32 vin_7n1 = 2;           // vin 码中第 7*n+1 个 ascii 编码
    uint32 vin_7n2 = 3;           // vin 码中第 7*n+2 个 ascii 编码
    uint32 vin_7n3 = 4;           // vin 码中第 7*n+3 个 ascii 编码
    uint32 vin_7n4 = 5;           // vin 码中第 7*n+4 个 ascii 编码
    uint32 vin_7n5 = 6;           // vin 码中第 7*n+5 个 ascii 编码
    uint32 vin_7n6 = 7;           // vin 码中第 7*n+6 个 ascii 编码
    uint32 vin_7n7 = 8;           // vin 码中第 7*n+7 个 ascii 编码
}

```

空调状态 2

数据内容	Key	值类型	备注
空调状态 2	ACM_INF2	ACM_INF2	

[复制代码](#)

```
message ACM_INF2 {  
    float acm_incar_temp = 1;    // 车内温度  
    float acm_outcar_temp = 2;   // 车外温度  
}
```

空调状态 4

数据内容	Key	值类型	备注
空调状态 4	ACM_INF4	ACM_INF4	

[复制代码](#)

```
message ACM_INF4 {  
    enum ACM_AirCond_WorkState {  
        POWER_OFF = 0;    // 关机  
        POWER_ON = 1;     // 开机  
        WORKSTATE_RESERVE = 2;    // 保留  
        WORKSTATE_UNAVAILABLE = 3; // 不可用  
    }  
    enum ACM_AirCond_ReqMode {  
        REQMODE_RESERVED = 0;    // 保留  
        AUTOMATIC_COOLING = 1;    // 自动制冷  
        ECO = 2;                  // ECO  
        VENTILATION = 3;          // 通风  
        REFRIGERATION = 4;        // 制冷  
        HEATING = 5;              // 制热  
        AUTOMATIC_HEATING = 6;    // 自动制热  
        REQMODE_UNAVAILABLE = 7;  // 不可用  
    }  
    enum ACM_EvaporFan_Speed {  
        EVAPORFAN_RESERVED_1 = 0; // 保留  
        GEAR_1 = 1;                 // 1 档  
        GEAR_2 = 2;                 // 2 档  
        GEAR_3 = 3;                 // 3 档  
        GEAR_4 = 4;                 // 4 档  
    }  
}
```

```

    GEAR_5 = 5;           // 5 档
    EVAPORFAN_RESERVED_2 = 6; // 保留
    EVAPORFAN_RESERVED_3 = 7; // 保留
}

ACM_AirCond_WorkState acm_aircond_workstate = 1; // 空调状态
ACM_AirCond_ReqMode acm_aircond_reqmode = 2;    // 空调工作模式
ACM_EvaporFan_Speed acm_evaporfan_speed = 3;    // 蒸发风机档位
float acm_currentset_temp = 4;                  // 当前设定温度
}

```

车辆加速度信息

数据内容	Key	值类型	备注
车辆加速度信息	YRS_INF	YRS_INF	



复制代码

```

message YRS_INF {
  enum Internal_Error {
    NO_INTERNAL_PROCESSING_ERROR = 0;
    INTERNAL_PROCESSING_ERROR = 1;
  }
  enum Voltage_Error {
    NO_OVERVOLTAGE_UNDERVOLTAGE = 0;
    OVERVOLTAGE_UNDERVOLTAGE = 1;
  }
  enum YRS_STAT {
    YRS_SIGNAL_SPECIFICATION_LIMITS = 0;
    YRS_TEMPORARY_SIGNAL_ERROR = 1;
    YRS_PERMANENT_SIGNAL_ERROR = 2;
    YRS_SENSOR_ELEMENT_INITIALISATION = 3;
  }
  enum AYS_STAT {
    AYS_SIGNAL_SPECIFICATION_LIMITS = 0;
    AYS_TEMPORARY_SIGNAL_ERROR = 1;
    AYS_PERMANENT_SIGNAL_ERROR = 2;
    AYS_SENSOR_ELEMENT_INITIALISATION = 3;
  }
  enum AXS_STAT {
    AXS_SIGNAL_SPECIFICATION_LIMITS = 0;
    AXS_TEMPORARY_SIGNAL_ERROR = 1;
    AXS_PERMANENT_SIGNAL_ERROR = 2;
  }
}

```

```
AXS_SENSOR_ELEMENT_INITIALISATION = 3;
}

uint32 CRC = 1;
Internal_Error internal_error = 2;
Voltage_Error voltage_error = 3;
YRS_STAT yrs_stat = 4;
AYS_STAT ays_stat = 5;
AXS_STAT axs_stat = 6;
float axs = 7;           // 纵向加速度
uint32 msg_cnt = 8;      // 报文计数
float ays = 9;           // 横向加速度
float yrs = 10;          // 横摆角速度
}
```

主目标信息

数据内容	Key	值类型	备注
主目标信息	PFC_Main_Obstacle_INF	PFC_Main_Obstacle_INF	

复制代码

```
message PFC_Main_Obstacle_INF {
    enum PFC_Main_Obstacle_Type {
        TYPE_INVALID = 0;           // 无效
        PEDESTRIAN = 1;             // 行人
        VEHICLE = 2;                 // 车辆
        TYPE_RESERVED_1 = 3;         // 预留
        TYPE_RESERVED_2 = 4;         // 预留
        TYPE_RESERVED_3 = 5;         // 预留
        TYPE_RESERVED_4 = 6;         // 预留
        TYPE_RESERVED_5 = 7;         // 预留
    }
    enum PFC_Camera_Connect_Fault {
        NO_CAMERA_CONNECT_FAULT = 0; // 无故障
        CAMERA_CONNECT_FAULT = 1;     // 有故障
    }
    enum PFC_Radar_Fault {
        NO_RADAR_FAULT = 0;           // 无故障
        RADAR_CONNECT_FAULT = 1;       // 连接故障
        RADAR_HARDWARE_FAULT = 2;      // 硬件故障
        RADAR_BLOCK_FAULT = 3;         // 堵塞故障
    }
}
```



```

    RADAR_CALIBRATION_FAULT = 4;    // 标定故障
    RADAR_NOT_WORKING = 5;          // 不工作
}
enum PFC_Vehicle_Connect_Fault {
    NO_VEHICLE_CONNECT_FAULT = 0;    // 无故障
    VEHICLE_CONNECT_FAULT = 1;       // 有故障
}
enum PFC_Fusion_Fault {
    NO_FUSION_FAULT = 0;              // 无故障
    FUSION_FAULT = 1;                 // 有故障
}

PFC_Main_Obstacle_Type pfc_main_obstacle_type = 1; // 主目标物体类型
float pfc_main_obstacle_pos_x = 2; // 距主目标纵向相对距离
float pfc_main_obstacle_pos_y = 3; // 距主目标横向相对距离
float pfc_main_obstacle_rel_vel_x = 4; // 与主目标纵向相对速度
float pfc_main_obstacle_rel_vel_y = 5; // 与主目标横向相对速度
PFC_Camera_Connect_Fault pfc_camera_connect_fault = 6;    // 摄像头连接
故障
PFC_Radar_Fault pfc_radar_fault = 7;                      // 雷达故障
PFC_Vehicle_Connect_Fault pfc_vehicle_connect_fault = 8;  // 车辆连接故
障
PFC_Fusion_Fault pfc_fusion_fault = 9;                    // 感知融合故
障
}

```